Task 1: Logistic Regression

The dataset (26,048 train / 6,513 test) was preprocessed by replacing "?" entries with NaN, imputing numeric features using the median, and one-hot encoding categorical variables. Although *capital_gain* and *capital_loss* were listed as categorical in the instructions, their integer magnitude structure justified treating them as numeric. I trained a Logistic Regression model using lbfgs (max_iter=5000). Because the dataset is imbalanced (~76% earn ≤$50K), I trained two models: one with *class_weight='balanced'* and one without class weighting.

(b) Beta Coefficients

Logistic regression models the log-odds:

$$\log(P / 1-P) = \beta_0 + \beta^T x$$

Each coefficient represents the change in log-odds per unit increase in a feature, and the odds ratio is exp(β). The largest positive coefficient corresponded to *capital_gain*, indicating a strong association between investment gains and higher income. Being married and holding advanced degrees also showed strong positive effects. The largest negative coefficients included service-related occupations and *sex_Female*. These reflect statistical associations, not causal effects.

(c) Majority-Class Baseline

Always predicting the majority class (≤$50K) yields 75.9% accuracy. Both models must exceed this to demonstrate that meaningful patterns were learned from the features.

(d)/(e) Test Set Results

Model	Accuracy	Precision	Recall	F1	AUC
Majority-Class Baseline	0.759	—	—	—	—
Logistic (Balanced)	0.806	0.566	0.843	0.677	0.907
Logistic (Unbalanced)	0.855	0.744	0.607	0.669	0.907

Both models clearly outperform the baseline. Since both have nearly identical ROC-AUC, the main difference comes from how class weighting shifts the decision boundary. The balanced model achieves substantially higher recall, identifying more high-income individuals at the cost of more false positives. The unbalanced model is more conservative and achieves higher overall accuracy, but misses a larger share of the minority class. When correctly identifying high earners matters more than raw accuracy, the balanced model is the better choice.

Task 2 — Neural Network Regressor

For this task I built a neural network to predict house prices on the King County dataset. House prices are heavily right-skewed, so I applied a log transformation (np.log1p) to the target before training — this stabilizes variance and makes it easier for the network to learn. All numerical features were standardized using StandardScaler fitted only on the training split to avoid data leakage.

(a) Sigmoid vs. ReLU

I implemented a two-hidden-layer architecture (128 → 64 neurons) with a linear output layer and compared Sigmoid and ReLU activations while keeping all other hyperparameters fixed. With EarlyStopping, ReLU converged quickly but to a substantially worse validation loss. Sigmoid trained longer and achieved significantly better validation performance, so it was selected for the final model.

(b) Test Set Evaluation

After inverting predictions back to dollar scale with np.expm1, the final model achieves **MSE = $16.24B** and **$R^2$ = 0.8879**. The naive baseline (always predicting the mean price) gives $R^2 \approx 0$, so explaining nearly 89% of the variance is a meaningful result.

Task 3 — Neural Network Classifier (CIFAR-10)

I built a multi-class MLP classifier using CIFAR-10 (60,000 color images, 10 classes). Images were normalized to [0, 1] with ToTensor() and flattened into 3,072-dimensional vectors (32×32×3) before being fed into the network, as required by the assignment.

(a) Architecture Search

I systematically compared shallow-wide (e.g., 2×512), deep-narrow (e.g., 6×128), and deep-medium (e.g., 6×256) configurations, keeping optimizer, learning rate, batch size, and epochs fixed. The best validation accuracy came from the shallow-wide architecture. Increasing depth did not improve performance and often slowed convergence — consistent with the fact that flattened pixel vectors do not preserve spatial structure, so additional depth does not provide hierarchical feature learning benefits.

(b) Regularization

I evaluated Dropout (p = 0.3 and p = 0.5) and L2 regularization (weight_decay = 1e-4) on the best architecture. L2 provided the most stable validation improvement and reduced the train–validation loss gap more effectively than dropout. Higher dropout rates reduced overfitting but also degraded overall accuracy, likely because aggressive dropout reduces effective capacity in an already capacity-limited MLP operating on raw pixel features.

(c) Final Test Evaluation

After a final training phase (35 epochs, lr = 5×10^{-4}) with L2 regularization, the selected model achieved **~52% test accuracy** and a **cross-entropy loss of ~1.70**. The confusion matrix reveals systematic misclassification between visually similar animal categories, particularly dog and cat. Classes with distinctive global shapes, such as ship and frog, achieved higher accuracy, while cat was among the hardest classes.

Overall performance aligns with expectations for fully connected networks on CIFAR-10. By flattening images, spatial relationships between neighboring pixels are destroyed, preventing the model from learning local edge patterns or hierarchical visual features. Convolutional Neural Networks (CNNs), which preserve spatial structure, typically achieve substantially higher accuracy on this dataset. Therefore, the main limitation observed here is architectural rather than due to optimization choices.

Appendix — Figures & Tables

All figures and tables below come directly from the three submitted notebooks. Each entry indicates the exact cell to screenshot and paste here.

A — Task 1: Logistic Regression

Table A1 — Top Beta Coefficients (Balanced Model)

Top 10 positive and top 10 negative beta coefficients with their corresponding odds ratios for the balanced model.

```
Top 10 features with the largest POSITIVE weights (increase odds of >50K):
```

	Feature	Beta (Coefficient)	Odds Ratio = exp(Beta)
2	capital_gain	2.267893	9.659030
30	marital_status_Married-AF-spouse	1.663265	5.276509
63	native_country_Cambodia	1.255660	3.510155
31	marital_status_Married-civ-spouse	1.244109	3.469842
84	native_country_Italy	0.879739	2.410271
55	relationship_Wife	0.854989	2.351349
103	native_country_Yugoslavia	0.824328	2.280348
39	occupation_Exec-managerial	0.750843	2.118785
27	education_Prof-school	0.650506	1.916510
23	education_Doctorate	0.650077	1.915689

```
Top 10 features with the largest NEGATIVE weights (decrease odds of >50K):
```

	Feature	Beta (Coefficient)	Odds Ratio = exp(Beta)
44	occupation_Priv-house-serv	-1.963356	0.140386
61	sex_Female	-1.606080	0.200673
66	native_country_Columbia	-1.585650	0.204815
33	marital_status_Never-married	-1.402542	0.245971
53	relationship_Own-child	-1.371912	0.253622
68	native_country_Dominican-Republic	-1.286299	0.276291
12	workclass_Without-pay	-1.222102	0.294610
34	marital_status_Separated	-1.176352	0.308402
26	education_Preschool	-1.176070	0.308489
40	occupation_Farming-fishing	-1.170983	0.310062

Table A2 — Performance Comparison (All Models)

Model performance comparison table: Accuracy, Precision, Recall, F1-score, and ROC-AUC for the Baseline, Balanced, and Unbalanced models.

	Model	Accuracy	Precision (Class=1)	Recall (Class=1)	F1-score	ROC-AUC
0	Baseline (Majority Class)	0.759291	NaN	NaN	NaN	NaN
1	Logistic (Balanced)	0.806080	0.565812	0.842775	0.677065	0.907307
2	Logistic (Unbalanced)	0.854752	0.743570	0.607257	0.668535	0.907144

B — Task 2: Neural Network Regressor

Figure B1 — Price Distribution Before and After log1p

Side-by-side histograms showing the original right-skewed price distribution and the normalized log1p(price) distribution after transformation.

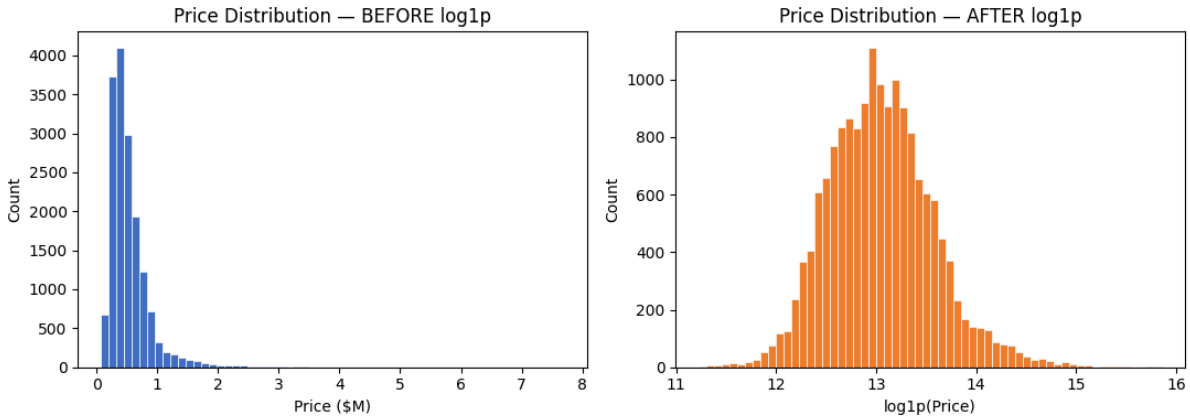


Table B2 — Activation Comparison Summary

Activation comparison summary: activation function, epochs to early stopping, and best validation loss (log-scale MSE) for Sigmoid and ReLU.

	Activation	Epochs (to early stop)	Best val_loss (log MSE)
0	sigmoid	150	0.029863
1	relu	12	0.107623

Figure B3 — Training & Validation Loss: Sigmoid vs. ReLU

Side-by-side training and validation loss curves for both activations. Sigmoid converges later and achieves substantially lower validation loss than ReLU.

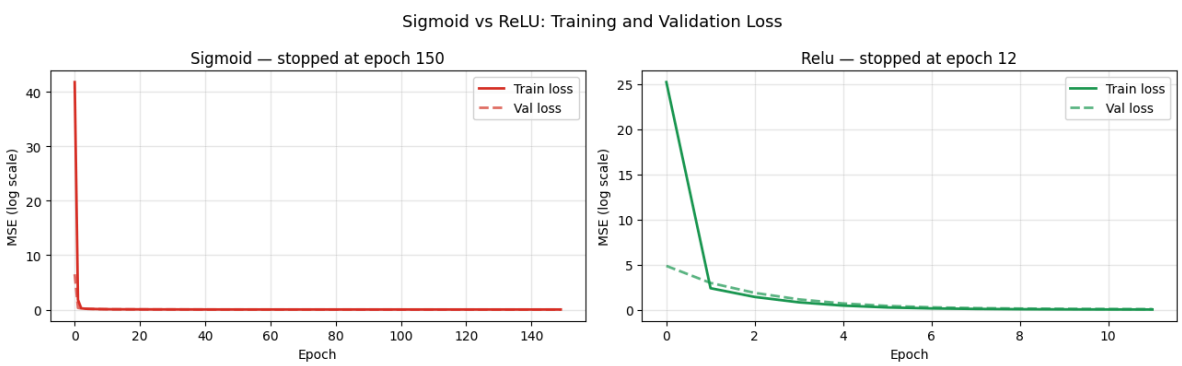
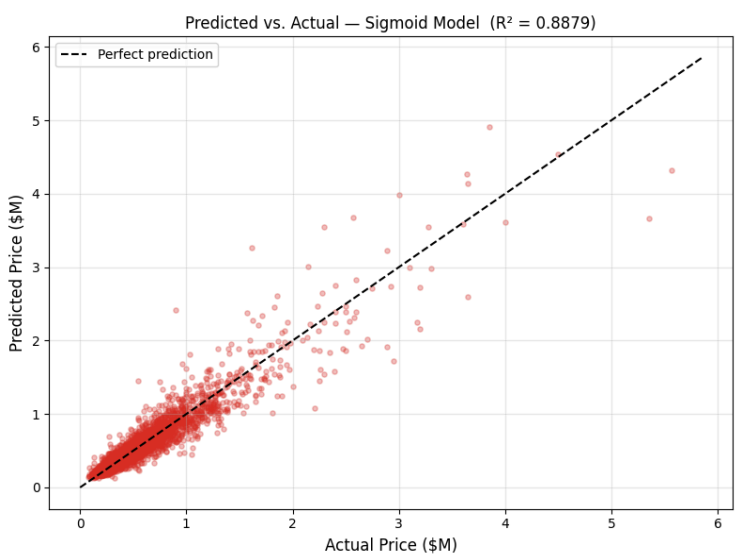


Figure B4 — Predicted vs. Actual House Prices

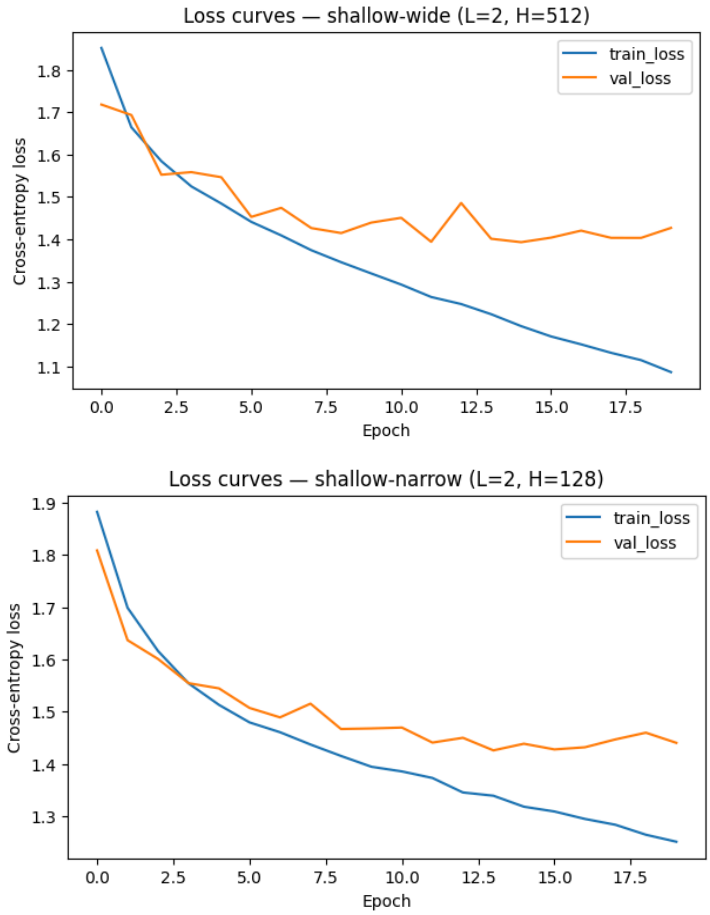
Scatter plot of actual vs. predicted prices (in $M) on the test set, with a perfect-prediction diagonal. Demonstrates strong model calibration.

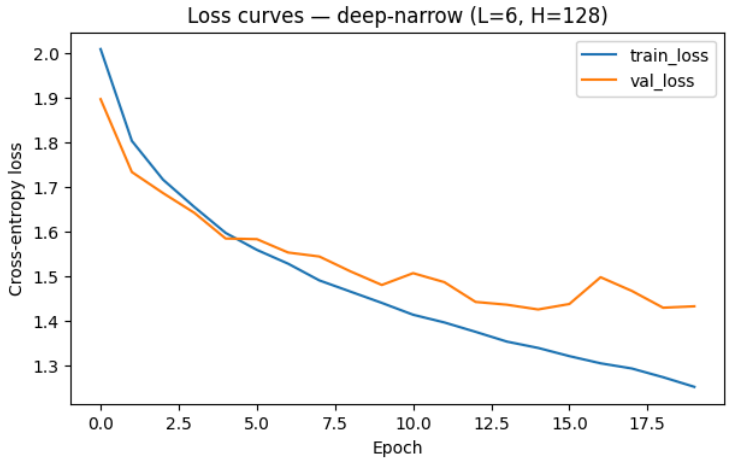


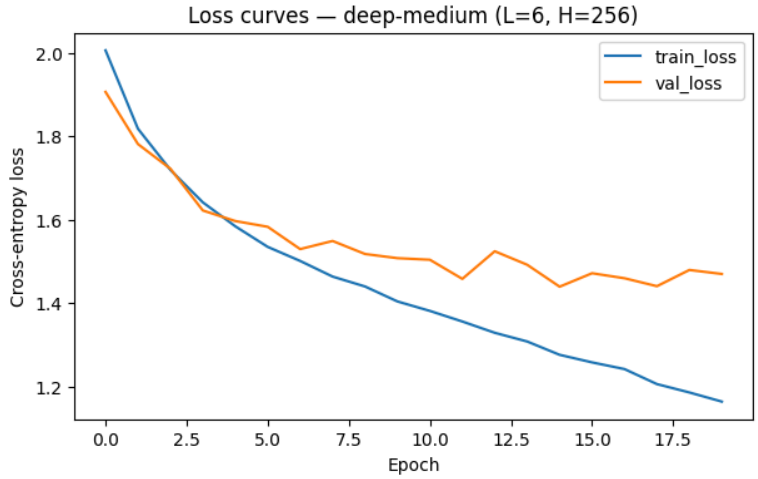
C — Task 3: Neural Network Classifier

Figure C1 — Architecture Experiments: Loss Curves

Training and validation loss curves for all tested architectures over 20 epochs.







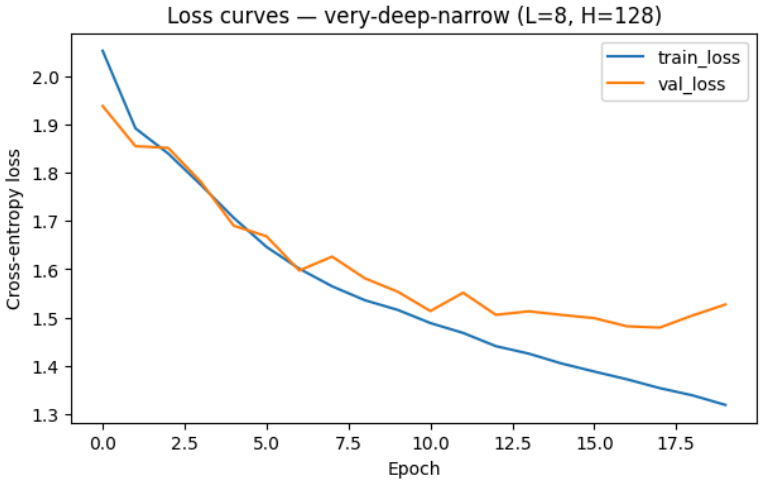
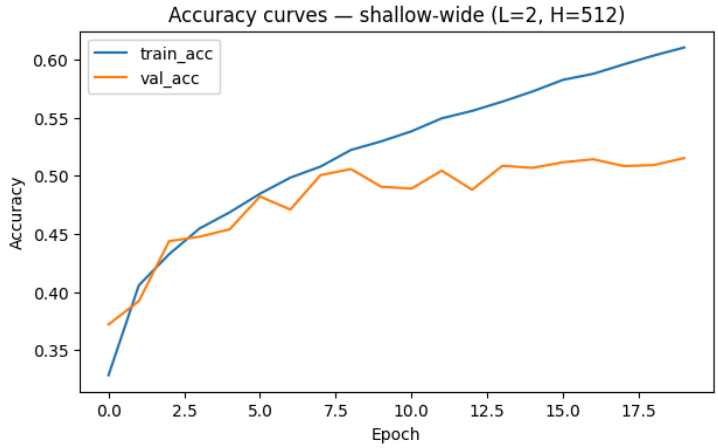
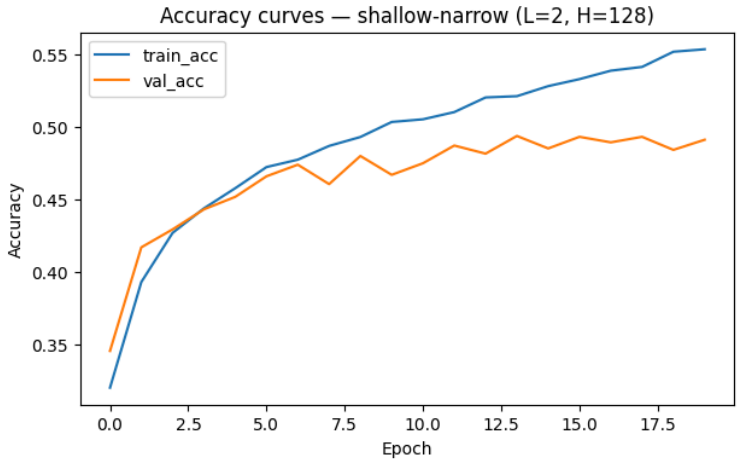
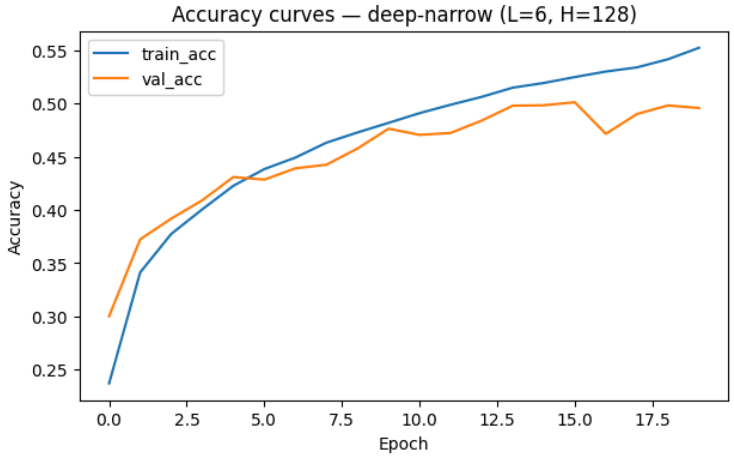
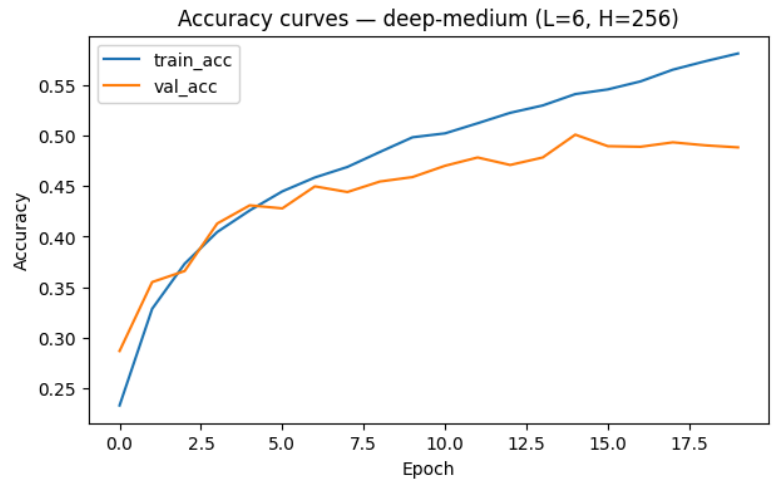


Figure C2 — Architecture Experiments: Accuracy Curves
Training and validation accuracy curves for all tested architectures. Confirms shallow-wide achieves the best validation accuracy; very-deep-narrow converges slowest.


Accuracy curves — shallow-wide (L=2, H=512)
train_acc
val_acc
Accuracy
Epoch
0.60
0.55
0.50
0.45
0.40
0.35
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Accuracy curves — shallow-narrow (L=2, H=128)
train_acc
val_acc
Accuracy
Epoch
0.55
0.50
0.45
0.40
0.35
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Accuracy curves — deep-narrow (L=6, H=128)
train_acc
val_acc
Accuracy
Epoch
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5



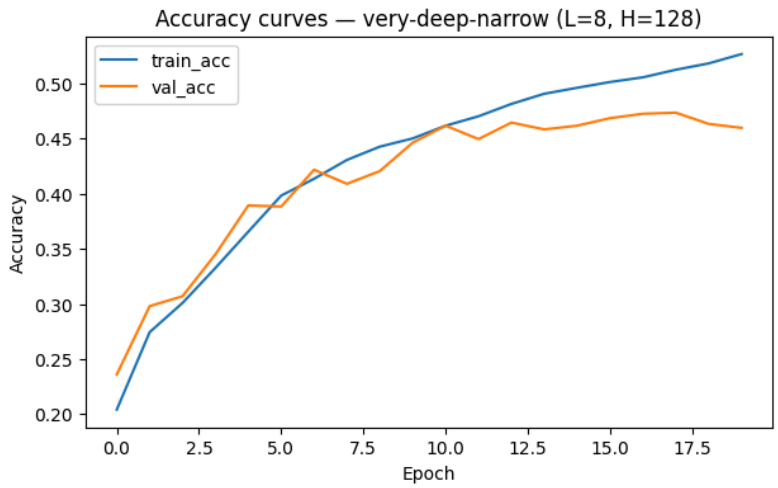


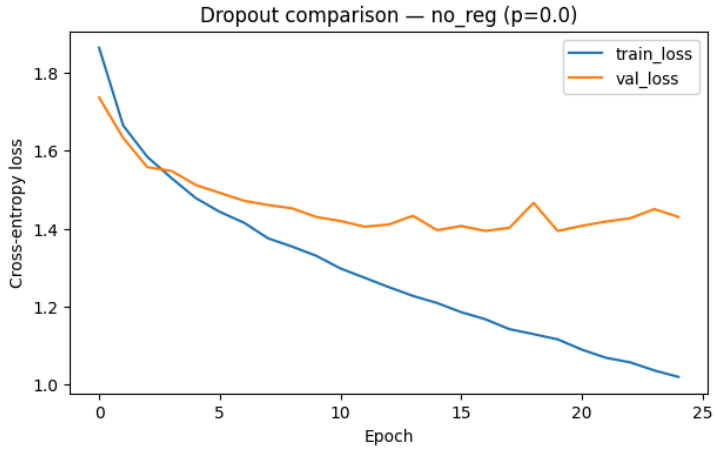
Table C1 — Architecture Comparison Summary

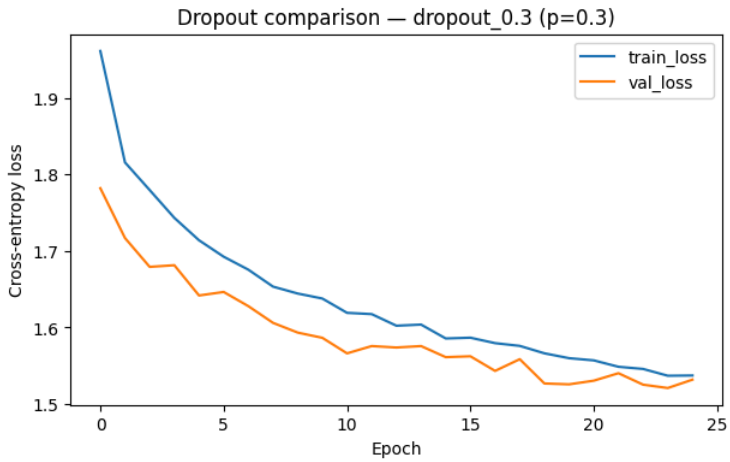
Architecture comparison summary: hidden layers, hidden units, training time, best validation accuracy, best validation loss, and train–validation loss gap for each configuration.

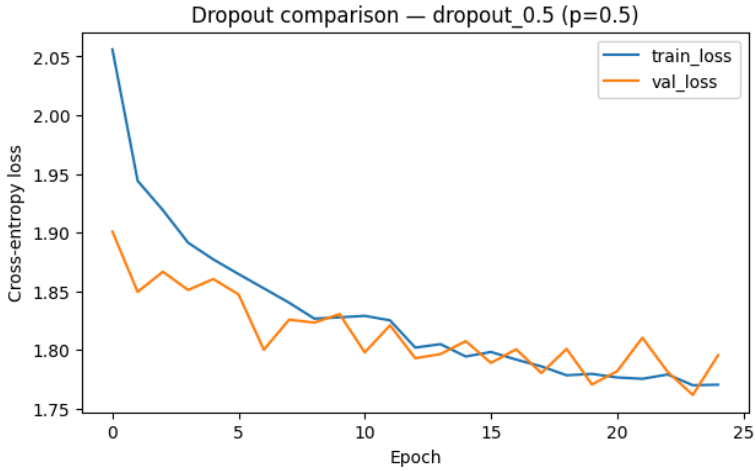
	name	hidden_layers	hidden_units	dropout	weight_decay	time_sec	best_val_acc	best_val_loss	final_train_loss	final_val_loss	train_val_loss_gap
0	shallow-wide	2	512	0.0	0.0	151.100290	0.5152	1.392942	1.086351	1.426683	0.340332
2	deep-narrow	6	128	0.0	0.0	151.134698	0.5010	1.424539	1.250789	1.431658	0.180869
3	deep-medium	6	256	0.0	0.0	151.191247	0.5010	1.439602	1.164100	1.470127	0.306027
1	shallow-narrow	2	128	0.0	0.0	145.535663	0.4938	1.425987	1.251331	1.440409	0.189078
4	very-deep-narrow	8	128	0.0	0.0	153.034101	0.4736	1.479389	1.319411	1.527075	0.207664

Figure C3 — Regularization Comparison: Loss Curves

Training and validation loss curves for all regularization configurations. A smaller gap between train and validation lines indicates more effective regularization.


Dropout comparison — no_reg (p=0.0)
train_loss
val_loss
Cross-entropy loss
Epoch
1.8
1.6
1.4
1.2
1.0
0
5
10
15
20
25


Dropout comparison — dropout_0.3 (p=0.3)
train_loss
val_loss
Cross-entropy loss
Epoch
1.9
1.8
1.7
1.6
1.5
0
5
10
15
20
25


Dropout comparison — dropout_0.5 (p=0.5)
train_loss
val_loss
Cross-entropy loss
Epoch
2.05
2.00
1.95
1.90
1.85
1.80
1.75
0
5
10
15
20
25

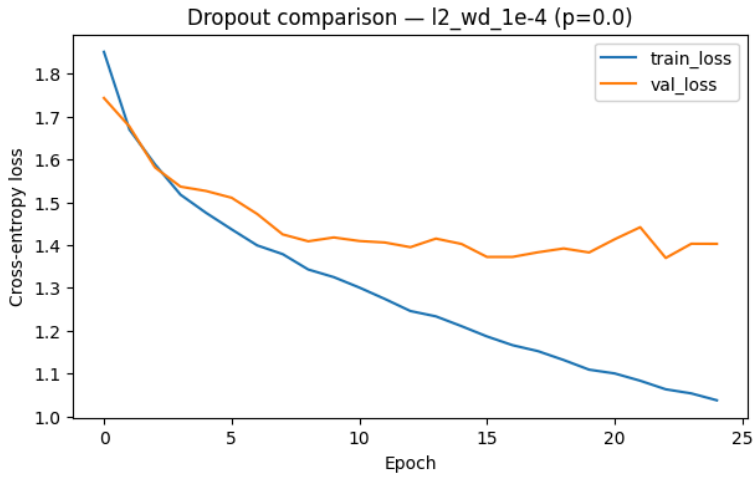
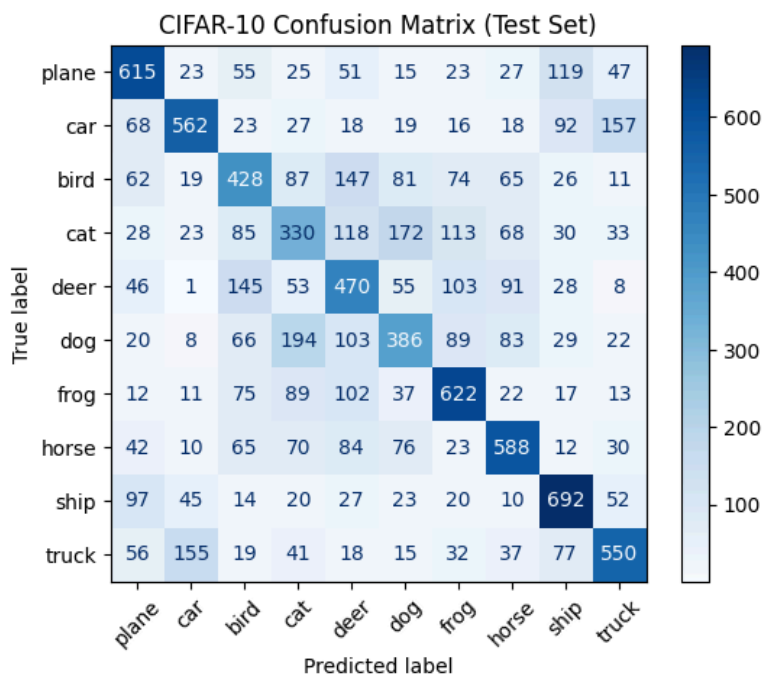


Table C2 — Regularization Summary

Regularization summary: dropout rate, weight decay, final train/validation loss, train–validation gap, and best validation accuracy for each strategy.

	name	dropout	weight_decay	time_sec	final_train_loss	final_val_loss	train_val_loss_gap	best_val_acc	model_state
0	no_reg	0.0	0.0000	185.985776	1.019539	1.430041	0.410502	0.5268	{'net.0.weight': [[tensor(0.0255, device='cuda...
1	dropout_0.3	0.3	0.0000	185.569312	1.537281	1.531830	-0.005451	0.4696	{'net.0.weight': [[tensor(-0.0198, device='cud...
2	dropout_0.5	0.5	0.0000	185.987225	1.770410	1.795582	0.025172	0.4078	{'net.0.weight': [[tensor(-0.0201, device='cud...
3	l2_wd_1e-4	0.0	0.0001	185.029209	1.037353	1.402742	0.365389	0.5332	{'net.0.weight': [[tensor(-1.8377e-20, device=...

Figure C4 — CIFAR-10 Confusion Matrix (Final Model, Test Set)

10×10 ConfusionMatrixDisplay heatmap. X-axis = predicted class, Y-axis = true class. Diagonal = correct predictions. Highlights the most and least confused class pairs.